

Homework 4

November 19, 2024

```
[7]: ## Import Statement
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import sklearn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn import metrics
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

0.1 Problem 1)

0.1.1 1a)

```
[2]: df = pd.read_csv('fifa22.csv')
print(df.head(5))
```

	name	rank	gender	wage_eur	log_wage	position	\
0	Lionel Andrés Messi Cuccittini	93	M	320000.0	12.676076	RW	
1	Lucia Roberta Tough Bronze	92	F	NaN	NaN	NaN	
2	Vivianne Miedema	92	F	NaN	NaN	NaN	
3	Wendéleine Thérèse Renard	92	F	NaN	NaN	NaN	
4	Robert Lewandowski	92	M	270000.0	12.506177	ST	

	nationality	club	league	preferred_foot	\
0	Argentina	Paris Saint-Germain	French Ligue 1	Left	
1	England	NaN	NaN	Right	
2	Netherlands	NaN	NaN	Right	
3	France	NaN	NaN	Right	
4	Poland	FC Bayern München	German 1. Bundesliga	Right	

	shooting	passing	dribbling	defending	attacking	skill	movement	power	\
--	----------	---------	-----------	-----------	-----------	-------	----------	-------	---

0	92.0	91.0	95.0	26.333333	85.8	94.0	90.2	77.8
1	61.0	70.0	81.0	89.000000	69.0	62.2	84.2	78.8
2	93.0	75.0	88.0	25.000000	86.0	79.0	80.6	84.0
3	70.0	62.0	73.0	91.333333	62.6	67.8	64.0	82.4
4	92.0	79.0	86.0	32.000000	86.0	81.4	81.6	84.8

	mentality	goalkeeping
0	73.833333	10.8
1	69.166667	12.6
2	70.833333	15.6
3	73.500000	12.8
4	80.666667	10.2

0.1.2 1b)

The unit of analysis of this data set appears to be conducted in an individual level.

0.1.3 1c)

The dimensions of this dataset are (19630,20), which means that there are 19630 observations and 20 features.

```
[3]: df.shape
```

```
[3]: (19630, 20)
```

0.1.4 1d)

There are 391 female players and 19239 male players in the dataset.

```
[4]: df_sex=df.groupby('gender').count()
df_sex['name']
```

```
[4]: gender
F      391
M    19239
Name: name, dtype: int64
```

0.1.5 1e)

```
[5]: df=df.dropna(0,subset=['passing'])
df.shape
```

```
[5]: (17450, 20)
```

0.2 Problem 2

0.2.1 2a)

```
[11]: model = smf.ols('rank ~ passing + attacking + defending + skill', data=df).fit()

model.summary()
```

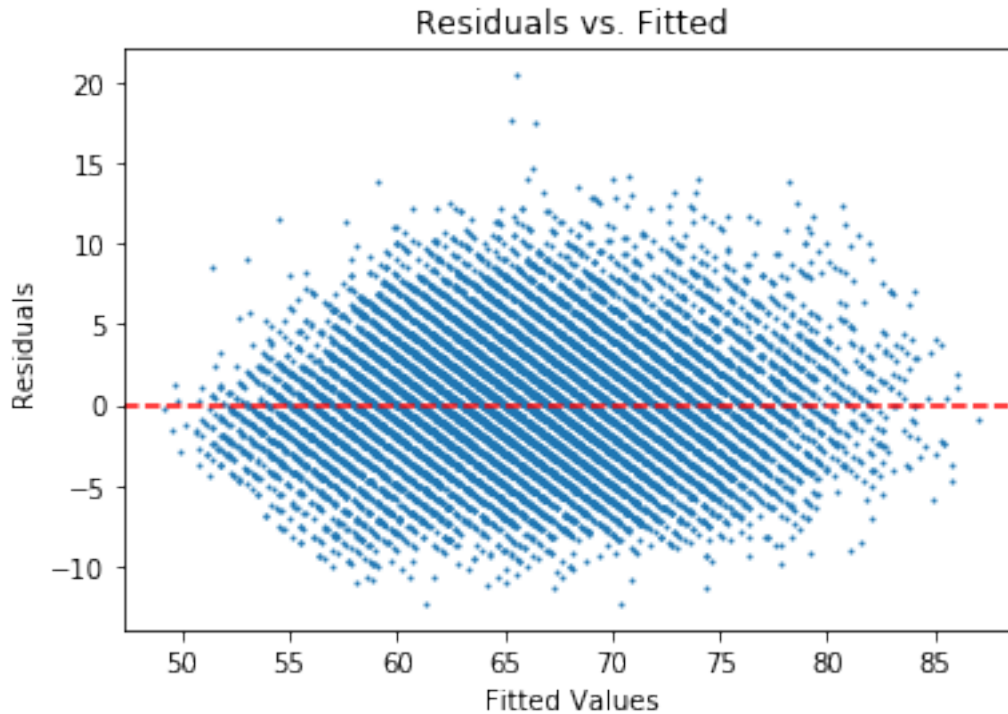
```
[11]: <class 'statsmodels.iolib.summary.Summary'>
      """
                OLS Regression Results
=====
Dep. Variable:          rank    R-squared:                0.705
Model:                  OLS    Adj. R-squared:            0.705
Method:                 Least Squares    F-statistic:        1.044e+04
Date:                  Tue, 19 Nov 2024    Prob (F-statistic):    0.00
Time:                  09:29:19    Log-Likelihood:        -47856.
No. Observations:      17450    AIC:                  9.572e+04
Df Residuals:          17445    BIC:                  9.576e+04
Df Model:               4
Covariance Type:       nonrobust
=====
                coef    std err          t      P>|t|      [0.025      0.975]
-----
Intercept          25.3278      0.203     124.785      0.000      24.930      25.726
passing            -0.0247      0.010      -2.425      0.015      -0.045      -0.005
attacking           0.6109      0.006      94.005      0.000       0.598       0.624
defending           0.1719      0.002      84.413      0.000       0.168       0.176
skill               0.0066      0.009       0.730      0.465      -0.011       0.024
=====
Omnibus:             171.799    Durbin-Watson:        1.342
Prob(Omnibus):        0.000    Jarque-Bera (JB):     178.339
Skew:                 0.234    Prob(JB):              1.88e-39
Kurtosis:             3.163    Cond. No.              790.
=====

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
      """
```

0.2.2 2b)

There are no significant problems shown on the residual vs. fitted graph since all value seems to be randomly spread out, which means that there are no clear indications of trend, suggesting that our model is appropriate.

```
[12]: residual = model.resid
      fitted = model.fittedvalues
      plt.scatter(fitted,residual,s=1)
      plt.axhline(y=0, color='red', linestyle='--')
      plt.xlabel('Fitted Values')
      plt.ylabel('Residuals')
      plt.title('Residuals vs. Fitted')
      plt.show()
```



0.2.3 2c)

About 70.5% of the variations in rank are explained by our model.

0.2.4 2d)

For every one unit increase in skill, the rank of that player increases by 0.0066

0.2.5 2e)

If we run our study many times, we are 95% confident that the true coefficient lies between -0.011 and 0.024. This means that for every one-unit increase in skill, the rank of that player increases by somewhere between -0.011 and 0.024.

0.2.6 2f)

Because the null hypothesis suggests that the independent variables have no effect on the dependent variable, which means that the coefficient of those independent variables is 0. Since the confidence interval of skill's coefficient contains 0, suggesting that when other independent variables are held as constant, it is possible that the true effect of skill on rank could be 0. This could be interpreted as under the 95% confidence level, skill could have no effect on ranking of the player.

0.3 Problem 3

0.3.1 3a)

Based on the R-squared provided by the summary, I think the four features will do a pretty good job predicting the ranking of a player since it could explain around 70.5% of the data. Although from the past two questions we conclude that skill might not be a good explanatory feature for rank in the multiple linear regression because of the fact that the confidence interval includes zero, which suggests that it is possible that the null hypothesis could be true, the regression still explained a lot of the data and should be able to make good prediction.

0.3.2 3b)

```
[13]: x = df[['passing', 'attacking', 'defending', 'skill']]
      y= df[['rank']]
      print(x.head())
      print(y.head())
```

	passing	attacking	defending	skill
0	91.0	85.8	26.333333	94.0
1	70.0	69.0	89.000000	62.2
2	75.0	86.0	25.000000	79.0
3	62.0	62.6	91.333333	67.8
4	79.0	86.0	32.000000	81.4

	rank
0	93
1	92
2	92
3	92
4	92

0.3.3 3c)

```
[14]: x_train,x_validation,y_train,y_validation = train_test_split(x,y,test_size=0.
      ↪25,random_state=123)
      print(x_train.head())
```

	passing	attacking	defending	skill
17226	52.0	48.0	59.333333	53.2
13548	48.0	55.0	12.666667	54.0
17874	59.0	46.2	58.000000	57.8

19599	47.0	40.6	46.666667	40.0
15629	49.0	51.8	25.666667	49.6

0.3.4 3d)

```
[15]: model = LinearRegression()
      result = model.fit(x_train,y_train)
      print(result.intercept_)
      print(result.coef_)
```

```
[25.16773306]
[[-0.02444506  0.61230756  0.17314968  0.00612364]]
```

0.3.5 3e)

Statsmodels package: Coefficients:(-0.0247,0.6109,0.1719,0.0066), Intercept:(25.3278).

SKLearn LinearRegression : Coefficients:(-0.02444506 0.61230756 0.17314968 0.00612364), Intercept: (25.16773306)

The coefficient of “attacking” estimated by statsmodels linear regression model using the full dataset is slightly lower than the coefficient of “attacking” estimated by the SKLearn linear regression model using 25% of the dataset. Specifically, the estimation generated by the statsmodels is 0.0014 lower than the estimation generated by the SKLearn linear regression model, which means that for every one unit increase in skill, the increase in ranking by the SKLearn model is approximately 0.0014 higher than the ranking in statsmodels(if all other variable are treated as a constant).

0.3.6 3f)

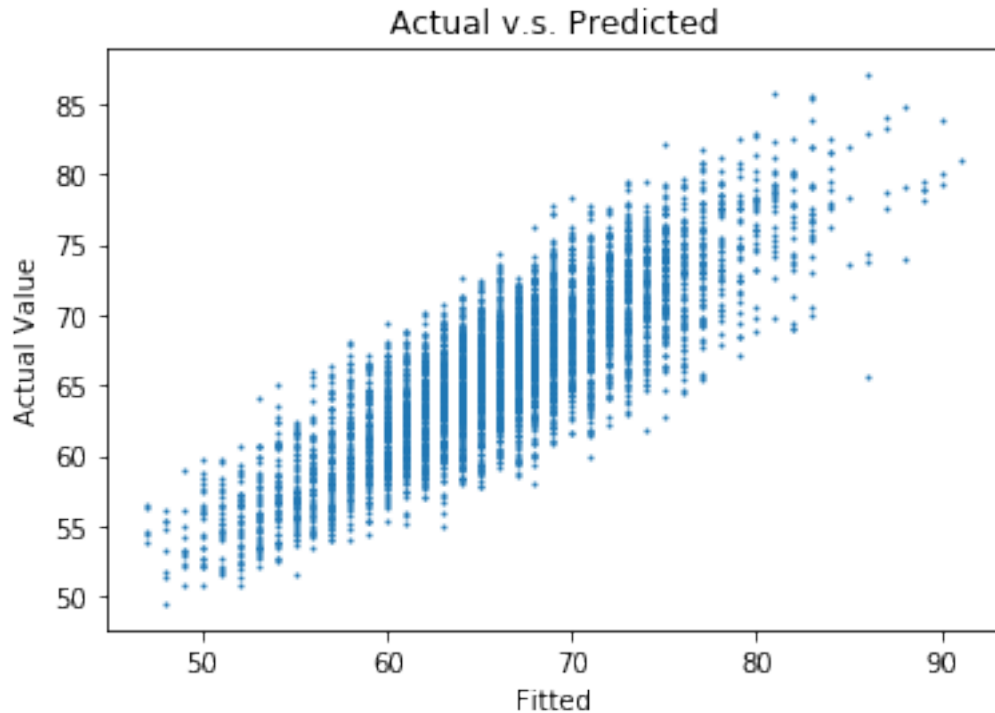
```
[16]: predictions = result.predict(x_validation)
      print(predictions[0:3])
```

```
[[64.57617047]
 [72.78035994]
 [70.46341746]]
```

0.3.7 3g)

```
[17]: plt.scatter(y_validation,predictions,s=1)
      plt.xlabel('Fitted')
      plt.ylabel('Actual Value')
      plt.title('Actual v.s. Predicted')
```

```
[17]: Text(0.5, 1.0, 'Actual v.s. Predicted')
```



0.3.8 3h)

```
[18]: rmse = np.sqrt(mean_squared_error(predictions,y_validation))
      print(rmse)
```

3.744562639987198

0.3.9 3i)

I think the model actually did a pretty good job because first of all with only 25% of the data, the intercept and coefficients appear to be very similar to that of the model trained by the full dataset. Second, the RMSE is relatively small, suggesting that the model predict the ranking with a little error. Third, the graph of Actual v.s. Predicted shows a nice linear trend along the $y=x$ line, which means that the dataset is not heavily biased.

0.4 Problem 4

0.4.1 4a)

```
[19]: df_preferred = df['preferred_foot'].value_counts()
      print(df_preferred.head())
```

```
Right    13044
Left     4406
```

Name: preferred_foot, dtype: int64

0.4.2 4b)

```
[20]: df_preferred['Right']/df_preferred.sum()
```

```
[20]: 0.7475071633237822
```

0.4.3 4c)

```
[21]: Xc = df[['shooting', 'passing', 'dribbling', 'defending', 'attacking', 'skill', 'movement', 'power', 'mentality', 'goalkeeping']]
print(Xc[0:5])
```

	shooting	passing	dribbling	defending	attacking	skill	movement	power	\
0	92.0	91.0	95.0	26.333333	85.8	94.0	90.2	77.8	
1	61.0	70.0	81.0	89.000000	69.0	62.2	84.2	78.8	
2	93.0	75.0	88.0	25.000000	86.0	79.0	80.6	84.0	
3	70.0	62.0	73.0	91.333333	62.6	67.8	64.0	82.4	
4	92.0	79.0	86.0	32.000000	86.0	81.4	81.6	84.8	

	mentality	goalkeeping
0	73.833333	10.8
1	69.166667	12.6
2	70.833333	15.6
3	73.500000	12.8
4	80.666667	10.2

0.4.4 4d)

```
[22]: mean = Xc.mean()
std = Xc.std()

Xc_normalized = (Xc-mean)/std

print(Xc_normalized.head())
```

	shooting	passing	dribbling	defending	attacking	skill	movement	\
0	2.784232	3.296547	3.315263	-1.393009	3.400066	3.548479	2.774561	
1	0.597701	1.229684	1.876666	2.131606	1.593371	0.598192	2.072749	
2	2.854765	1.721794	2.595964	-1.468001	3.421574	2.156834	1.651663	
3	1.232501	0.442307	1.054610	2.262841	0.905106	1.117739	-0.290015	
4	2.784232	2.115482	2.390450	-1.074294	3.421574	2.379497	1.768631	

	power	mentality	goalkeeping
0	1.944226	2.180551	0.281668
1	2.066442	1.623651	1.481058


```

2  2.701964    1.822544    3.480041
3  2.506419    2.140773    1.614323
4  2.799737    2.996013   -0.118129

```

0.4.5 4e)

```

[23]: x_train,x_val,y_train,y_val = \
      ↪ train_test_split(Xc_normalized,df['preferred_foot'],test_size=0.
      ↪ 3,random_state=456)
      print(x_train[0:3])

```

```

      shooting    passing    dribbling    defending    attacking    skill \
17219 -2.012028 -1.427712 -1.514313    0.444290 -1.826444 -1.572774
10931 -0.460297  0.343885  0.746339    0.425542 -0.277849  0.616747
13667  0.315569  0.147041  0.129798   -0.886815  0.087792  0.004423

      movement    power    mentality    goalkeeping
17219 -0.968432 -1.013395 -1.558639    0.548199
10931  0.692521 -0.646748 -0.305613   -0.251395
13667 -0.056078 -0.402316 -0.365281   -1.850581

```

0.4.6 4f)

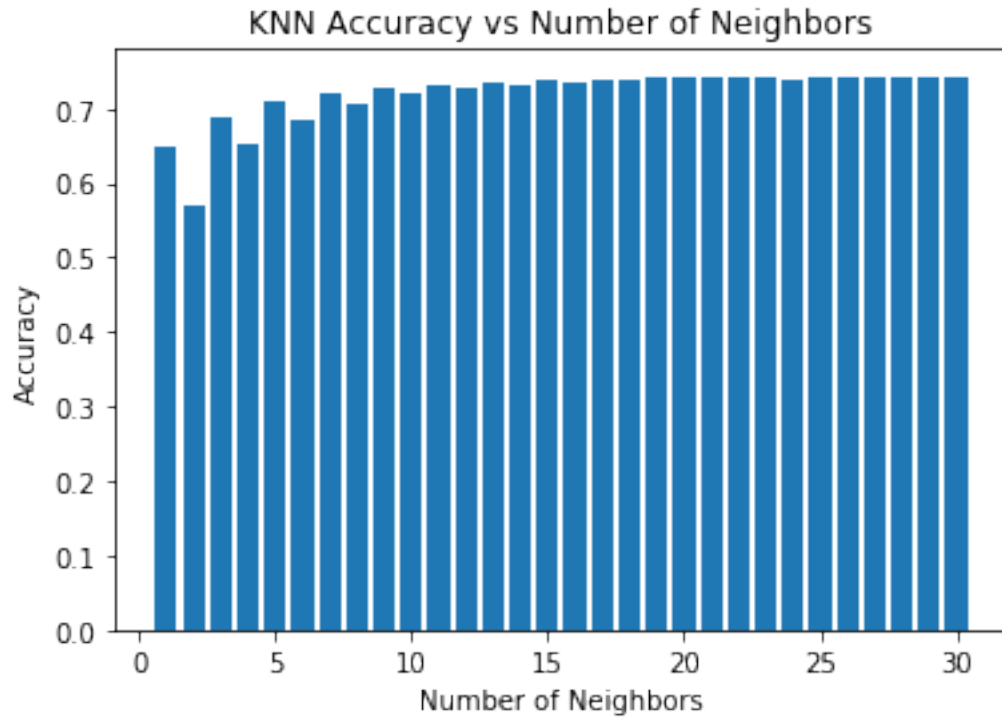
```

[28]: accuracies = []

      for i in range(1,31):
          knn = KNeighborsClassifier(n_neighbors=i)
          knn.fit(x_train, y_train)
          predictions = knn.predict(x_val)
          accuracies.append(accuracy_score(y_val, predictions))

      # Plot the results
      plt.bar(range(1,31), accuracies)
      plt.xlabel("Number of Neighbors")
      plt.ylabel("Accuracy")
      plt.title("KNN Accuracy vs Number of Neighbors")
      plt.show()

```



0.4.7 4g)

```
[25]: knn = KNeighborsClassifier(n_neighbors = 19)
      knn.fit(x_train,y_train)
      predictions = knn.predict(x_val)
      print(predictions[0:3])
```

```
['Right' 'Right' 'Right']
```

0.4.8 4h)

Approximately 1247 players who prefer left foot are classified as prefer right foot.

```
[26]: confusion_matrix(y_val,predictions)
```

```
[26]: array([[ 79, 1247],
           [ 98, 3811]])
```

0.4.9 4i)

The recall rate for Left being 0.06 suggests that out of all the true left foot players the model only correctly identified 6% of them. The recall rate suggest that our model is extremely bad at predicting the left foot players.

```
[27]: print(classification_report(y_val, predictions, target_names=["Left", "Right"]))
```

	precision	recall	f1-score	support
Left	0.45	0.06	0.11	1326
Right	0.75	0.97	0.85	3909
accuracy			0.74	5235
macro avg	0.60	0.52	0.48	5235
weighted avg	0.68	0.74	0.66	5235

0.4.10 4j)

The model is not doing a bad job overall since it has an overall accuracy of 0.74, but considering that the majority of the players preferred right foot, the model basically underestimate the number of left foot players to get a higher overall accuracy. I think different objectives influences how I evaluate the model. If the objective is to predict the overall accuracy of the preferred foot, then the model is doing okay under the assumption that the real population has a similar proportion of left foot to right foot player. If the objective is to identify the left foot player, then the model is doing a bad job because it only predicts 6% true left foot players.